Geavanceerde Computerarchitectuur Labo 3
Warps en thread divergence

Arno De Lathauwer
Indra Coppens

February 2025

1 Context

Dit labo bestond uit twee hoofdonderdelen. Ten eerste werd een afbeelding geïnverteerd om kennis te maken met warps, waarbij ook de invloed van het aantal threads per warp op de rekentijd werd onderzocht. Ten tweede werd er thread divergence gecreëerd door een langere operatie op de rode pixels uit te voeren. De uitvoeringstijd met thread divergence werd vervolgens vergeleken met de tijd zonder thread divergence.

2 Methodologie

Als eerste werd een GPU-kernel functie geschreven die een afbeelding inverteert. De afbeelding wordt ingeladen en omgezet naar een flattened array door de code voorzien op Toledo. Daarna wordt deze flattened array gekopieerd naar de memory van de GPU. Op de GPU zal iedere thread een nieuwe kleurwaarde berekenen via de formule: `new_image_array_d[idx] = (uint8_t)255 - image_array_d[idx]`. De bijhorende code is toegevoegd in Appendix A.

Vervolgens werd deze kernel functie aangepast om te kunnen werken met een aantal threads dat lager is dan het aantal elementen in de (flattened) image array, door middel van striding. Iedere thread verwerkt nu meerdere elementen door gebruik te maken van een for-loop in de kernel functie. Iedere thread verwerkt de elementen op de indexen `idx + k * (blockDim.x * gridDim.x)` (waarbij `k` gelijk is aan de k-de iteratie) 1. De code is toegevoegd in Appendix B.

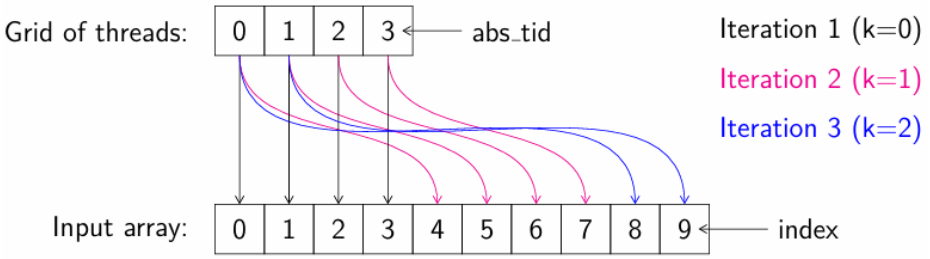


Figure 1: Part 1: Striding.

In het tweede deel van het labo werd voor iedere rood waarde van iedere pixel een andere bewerking uitgevoerd dan voor de groen en blauw waarden. Er werd namelijk gebruikt gemaakt van een 1D-gaussian blur voor de rood waarden. De groen en blauw waarden werden gewoon zoals in het eerste deel geïnverteerd. Deze verschillende bewerkingen werden mogelijk door te checken of het thread id (= `idx`) van iedere thread een veelvoud is van drie (aangezien in het formaat `rgbrgb...` de rood waarden telkens op een index staan die een veelvoud van drie is in de flattened image array). Indien dit zo is voeren we een andere bewerking uit. In deze functie treedt er thread divergence op. De code is toegevoegd in Appendix C.

Tot slot werd het tweede deel nog eens herhaald maar deze keer door gebruik te maken van een ander formaat: de image werd ingeladen en vervolgens omgezet naar een flattened image array van het formaat `rgbrgb`. Deze array werd vervolgens omgezet naar een array van het formaat `rrr...ggg...bbb...`, waarna opnieuw voor iedere rode waarde een 1D-gaussian blur werd bepaald en de groen en blauw waarden werden opnieuw gewoon geïnverteerd. In deze situatie treedt er geen thread divergence op. Deze code is toegevoegd in Appendix D.

3 Observaties/resultaten

3.1 Part 1: warps

In het eerste experiment werd een bewerking uitgevoerd waarbij iedere pixel in de afbeelding op dezelfde manier werd verwerkt. Hierdoor was de rekentijd uniform over alle pixels en trad er dus geen thread divergence op. We konden waarnemen dat de gecreëerde afbeelding het geïnverteerde was van de gegeven afbeelding. Een alternatieve implementatie met striding werd ook getest. Hierbij werden er minder threads gebruikt dan het aantal pixels in de afbeelding. Ook hier zagen we hetzelfde resultaat.

Vervolgens werd onderzocht hoe de rekentijd veranderde wanneer het aantal threads per blok werd aangepast. Een warp zit binnen één blok en overlapt dus niet over verschillende blokken. Wanneer we twee blokken hebben met elk 16 threads, dan wil dit zeggen dat elke blok één warp zal schedulen waarbij 16 van de 32 threads gebruikt zijn en 16 threads idle zijn. Op die manier konden we het effect testen van het gedeeltelijk of volledig gebruik van warps.

Uit de experimenten bleek een duidelijke daling in de rekentijd naarmate het aantal (non-idle) threads per warp toenam, dit is te zien in *Figuur 2*. Dit komt doordat een groter aantal threads binnen een warp actief is, waardoor er minder threads idle moeten wachten en er dus meer werk in parallel wordt uitgevoerd. Intuïtief zou men verwachten dat 32 threads per blok optimaal is, aangezien een warp dan volledig gevuld is. Echter, uit de metingen bleek dat de rekentijd verder bleef afnemen bij grotere aantallen threads per blok.

De verklaring hiervoor ligt in de architectuur van de gebruikte GPU. De processor (van de labo-computer) kan zes warps parallel uitvoeren, maar beschikt slechts over twee streaming multiprocessors (SM's), waardoor er telkens slechts twee blokken tegelijkertijd uitgevoerd konden worden. Wanneer het aantal threads per blok verhoogd werd naar bijvoorbeeld 64 threads per blok, waren er minder blokken nodig om de volledige afbeelding te verwerken. Doordat er telkens twee blokken parallel kunnen worden uitgevoerd en er minder blokken in totaal nodig zijn daalt de tijd verder ten opzichte van wanneer er 32 threads per blok werden gescheduled. Naarmate het aantal threads verder toenam, begon de rekentijd te stagneren. Op dit punt zijn de limieten van de hardware bereikt, waarbij extra threads geen verdere prestatiewinst opleveren.

3.2 Part 2: thread divergence

In het tweede deel werd onderzocht hoe thread divergence de rekentijd beïnvloedt. We introduceerden thread divergence door een complexere bewerking uit te voeren op de rode waarden dan op de groene en blauwe waarden van iedere pixel. Omdat threads binnen een warp gelijktijdig worden uitgevoerd, moesten de threads die groene en blauwe waarden verwerkten wachten op de threads die rode waarden bewerkten, wat leidde tot thread divergence.

Om te kunnen vergelijken wat het effect was van deze thread divergence, vormden we het formaat van de image array om van '`rgbrgbrgb...`'-formaat naar '`rrr...ggg...bbb...`'-formaat. In het eerste geval is er thread divergence, zoals eerder vermeldt. In de tweede opstelling is er bijna geen thread divergence. Dit komt doordat eerst alle rode waarden worden verwerkt, daarna de groene en vervolgens de blauwe. De threads die de rode pixelwaarden verwerken worden dus niet gelijktijdig uitgevoerd met de threads die de andere kleuren verwerken (buiten in de overgang van rode pixelwaarden naar groene pixelwaarden is dit mogelijk). Hierdoor zijn de threads met een gelijke uitvoeringstijd gegroepeerd, waardoor ze niet moeten wachten op elkaar en er geen thread divergence is.

Als we nu beide formaten naast elkaar leggen en de tijd vergelijken dan zien we dat de uitvoering met thread divergence trager was dan de uitvoering zonder thread divergence (*Figuur 3*). Dit bevestigd ons vermoeden dat het efficiënter is om binnen eenzelfde warp de threads te verzamelen met gelijkaardige berekentijd, en dus thread divergence te minimaliseren.

4 Conclusie

In dit labo schreven we een programma dat een afbeelding inverteert en onderzochten we wat het effect is van het aantal threads per warp op de rekentijd. Daarnaast bekeken we het effect van thread divergence op de rekentijd. In het eerste deel zagen we dat een groter aantal actieve threads per warp de rekentijd verlaagt, doordat er minder idle threads zijn en de GPU dus efficiënter gebruikt wordt. In het tweede deel toonde de experimenten aan dat thread divergence de rekentijd verhoogt. Wanneer verschillende threads binnen een warp taken met sterk uiteenlopende rekentijden uitvoeren, leidt dit tot wachttijden en dus een minder efficiënte uitvoering. Door pixelwaarden met vergelijkbare rekentijden te groeperen, kon deze rekentijd verminderd worden.

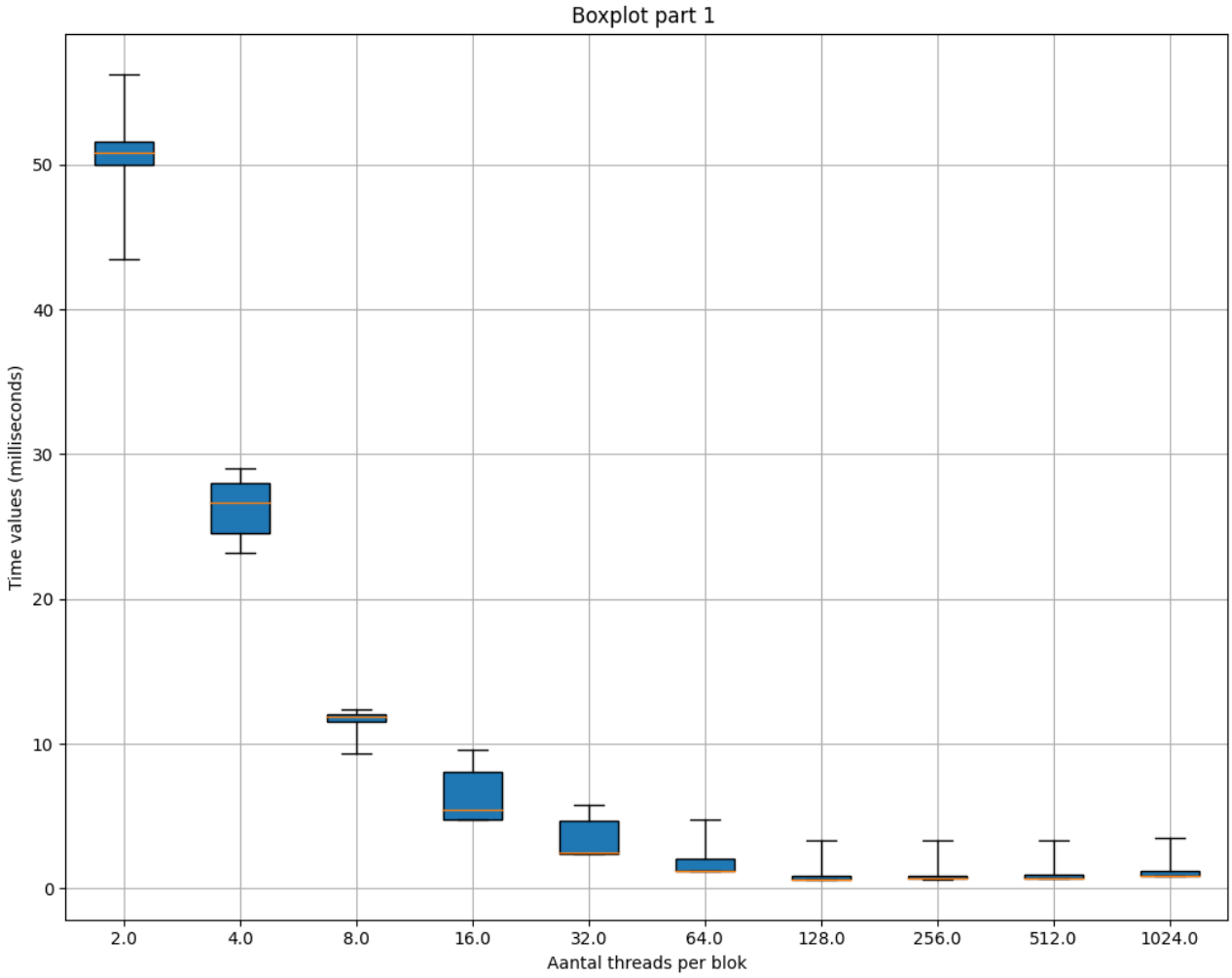


Figure 2: Part 1: Invloed van aantal threads (full/partial warps) op rekentijd.

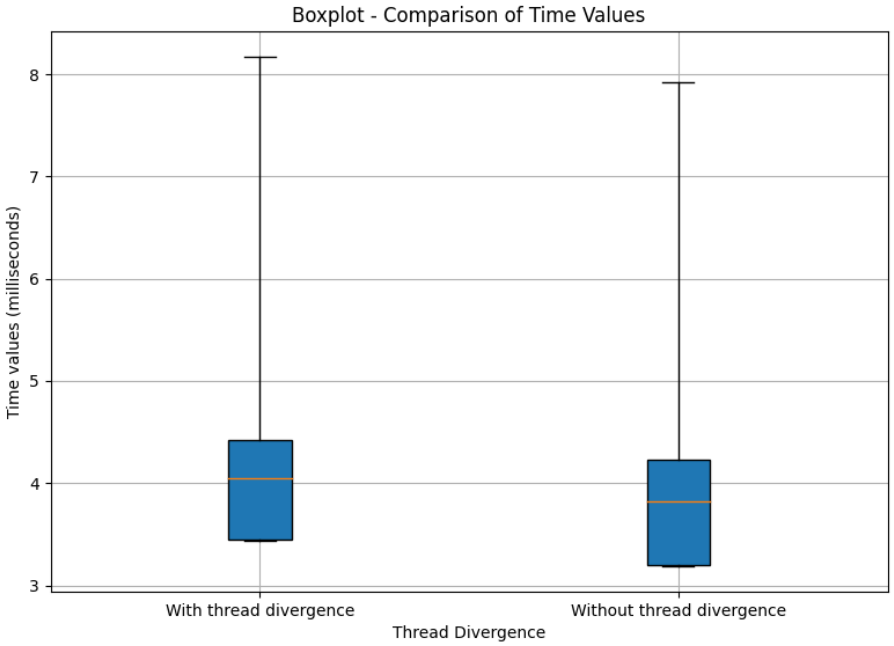


Figure 3: Part 2: Invloed van thread divergence op rekentijd.

Appendix A: Code part 1 (no striding)

```
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run `convert <name>.<extension> <name>.ppm
 *
 */
#include <cstdint>  // Data types
#include <iostream> // File operations
#define M 960      // VR width
#define N 1280     // VR height
#define C 3        // Colors
#define OFFSET 16 // Header length

uint8_t *get_image_array(void)
{
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Noets:
     *  - Images data is flattened per color, column, row.
     *  - The first 3 data elements are the RGB components
     *  - The first 3*M data elements represent the firts row of the image
     *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M}, r_{1,0}, ..., b_{b,M}, ..., b_{N, M}
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile = fopen("./anna.ppm", "rb");
    if (imageFile == NULL)
    {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Initialize empty image array
    uint8_t *image_array = (uint8_t *)malloc(M * N * C * sizeof(uint8_t) + OFFSET);

    // Read the image
    fread(image_array, sizeof(uint8_t), M * N * C * sizeof(uint8_t) + OFFSET, imageFile);

    // Close the file
    fclose(imageFile);
    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}

void save_image_array(uint8_t *image_array)
{
    /*
     * Save the data of an (RGB) image as a pixel map.
     *
     * Parameters:
     *  - param1: The data of an (RGB) image as a 1D array
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile = fopen("./output_image_ex1.ppm", "wb");
    if (imageFile == NULL)
    {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Configure the file
    fprintf(imageFile, "P6\n");           // P6 filetype
    fprintf(imageFile, "%d %d\n", M, N); // dimensions
    fprintf(imageFile, "255\n");          // Max pixel

    // Write the image
    fwrite(image_array, 1, M * N * C, imageFile);

```

```
    // Close the file
    fclose(imageFile);
}

__global__ void invertImage(uint8_t *image_array_d, uint8_t *new_image_array_d)
{
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    new_image_array_d[idx] = (uint8_t)255 - image_array_d[idx];

    __syncthreads();
}

int main(void)
{

    // Read the image
    uint8_t *image_array = get_image_array();

    // Allocate output
    uint8_t *new_image_array = (uint8_t *)malloc(M * N * C);

    uint8_t *image_array_d;
    uint8_t *new_image_array_d;

    // Invert
    cudaMalloc((void **)&image_array_d, M * N * C * sizeof(uint8_t));
    cudaMalloc((void **)&new_image_array_d, M * N * C * sizeof(uint8_t));

    cudaMemcpy(image_array_d, image_array, M *N * C * sizeof(uint8_t), cudaMemcpyHostToDevice);
    cudaMemcpy(new_image_array_d, new_image_array, M * N * C * sizeof(uint8_t),
        cudaMemcpyHostToDevice);

    invertImage<<<N*M*C/1024, 1024>>>(image_array_d, new_image_array_d);
    cudaDeviceSynchronize();

    cudaMemcpy(new_image_array, new_image_array_d, M * N * C * sizeof(uint8_t),
        cudaMemcpyDeviceToHost);
    // Save the image
    save_image_array(new_image_array);

    return 0;
}
```

Appendix B: Code part 1 (striding)

```
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run `convert <name>.<extension> <name>.ppm
 *
 */
#include <cstdint>  // Data types
#include <iostream> // File operations
#define M 960     // VR width
#define N 1280    // VR height
#define C 3       // Colors
#define OFFSET 16 // Header length

uint8_t *get_image_array(void)
{
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Noets:
     *   - Images data is flattened per color, column, row.
     *   - The first 3 data elements are the RGB components
     *   - The first 3*M data elements represent the firts row of the image
     *   - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M}, r_{1,0}, ..., b_{b,M}, ..., b_{N,
         M}
     *
     */
    // Try opening the file
    FILE *imageFile;
```

```
    imageFile = fopen("./anna.ppm", "rb");
    if (imageFile == NULL)
    {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Initialize empty image array
    uint8_t *image_array = (uint8_t *)malloc(M * N * C * sizeof(uint8_t) + OFFSET);

    // Read the image
    fread(image_array, sizeof(uint8_t), M * N * C * sizeof(uint8_t) + OFFSET, imageFile);

    // Close the file
    fclose(imageFile);
    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}

void save_image_array(uint8_t *image_array)
{
    /*
     * Save the data of an (RGB) image as a pixel map.
     *
     * Parameters:
     *  - param1: The data of an (RGB) image as a 1D array
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile = fopen("./output_image_ex1_striding.ppm", "wb");
    if (imageFile == NULL)
    {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Configure the file
    fprintf(imageFile, "P6\n");           // P6 filetype
    fprintf(imageFile, "%d %d\n", M, N); // dimensions
    fprintf(imageFile, "255\n");          // Max pixel

    // Write the image
    fwrite(image_array, 1, M * N * C, imageFile);

    // Close the file
    fclose(imageFile);
}

__global__ void invertImage(uint8_t *image_array_d, uint8_t *new_image_array_d)
{
    //int idx = threadIdx.x + blockIdx.x * blockDim.x;
    //new_image_array_d[idx] = (uint8_t)255 - image_array_d[idx];
    for(int idx = blockIdx.x * blockDim.x + threadIdx.x; idx < N*M*C; idx += blockDim.x * gridDim
        .x){
        new_image_array_d[idx] = (uint8_t)255 - image_array_d[idx];
    }

    __syncthreads();
}

int main(void)
{

    // Read the image
    uint8_t *image_array = get_image_array();

    // Allocate output
    uint8_t *new_image_array = (uint8_t *)malloc(M * N * C);

    uint8_t *image_array_d;
    uint8_t *new_image_array_d;

    // Invert
    cudaMalloc((void **)&image_array_d, M * N * C * sizeof(uint8_t));
    cudaMalloc((void **)&new_image_array_d, M * N * C * sizeof(uint8_t));

    cudaMemcpy(image_array_d, image_array, M *N * C * sizeof(uint8_t), cudaMemcpyHostToDevice);
```

```
    cudaMemcpy(new_image_array_d, new_image_array, M * N * C * sizeof(uint8_t),
        cudaMemcpyHostToDevice);

    invertImage<<<N*M*C/2048, 1024>>>(image_array_d, new_image_array_d);
    cudaDeviceSynchronize();

    cudaMemcpy(new_image_array, new_image_array_d, M * N * C * sizeof(uint8_t),
        cudaMemcpyDeviceToHost);
    // Save the image
    save_image_array(new_image_array);
    cudaFree(image_array_d);
    cudaFree(new_image_array_d);

    return 0;
}
```

Appendix C: Code part 2 (rgbrgb... format)

```
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run `convert <name>.<extension> <name>.ppm
 *
 */
#include <cstdint>  // Data types
#include <iostream> // File operations
#define M 960      // VR width
#define N 1280     // VR height
#define C 3        // Colors
#define OFFSET 16 // Header length

uint8_t *get_image_array(void)
{
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Noets:
     *  - Images data is flattened per color, column, row.
     *  - The first 3 data elements are the RGB components
     *  - The first 3*M data elements represent the firts row of the image
     *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M}, r_{1,0}, ..., b_{b,M}, ..., b_{N,
         M}
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile = fopen("./anna.ppm", "rb");
    if (imageFile == NULL)
    {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Initialize empty image array
    uint8_t *image_array = (uint8_t *)malloc(M * N * C * sizeof(uint8_t) + OFFSET);

    // Read the image
    fread(image_array, sizeof(uint8_t), M * N * C * sizeof(uint8_t) + OFFSET, imageFile);

    // Close the file
    fclose(imageFile);
    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}

void save_image_array(uint8_t *image_array)
{
    /*
     * Save the data of an (RGB) image as a pixel map.
     *
     * Parameters:
     *  - param1: The data of an (RGB) image as a 1D array
     *
```

```
     */
    // Try opening the file
    FILE *imageFile;
    imageFile = fopen("./output_image_ex2.ppm", "wb");
    if (imageFile == NULL)
    {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Configure the file
    fprintf(imageFile, "P6\n");          // P6 filetype
    fprintf(imageFile, "%d %d\n", M, N); // dimensions
    fprintf(imageFile, "255\n");         // Max pixel

    // Write the image
    fwrite(image_array, 1, M * N * C, imageFile);

    // Close the file
    fclose(imageFile);
}

__global__ void invertImage(uint8_t *image_array_d, uint8_t *new_image_array_d)
{
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if(idx%3 == 0){
        if(6 < idx < M*N - 6){
            new_image_array_d[idx] = (uint8_t) 0.1*image_array_d[idx-6] + 0.25 * image_array_d[
                idx -3] + 0.5 * image_array_d[idx] + 0.25 *image_array_d[idx + 3] + 0.1*
                image_array_d[idx + 6];
        }
    }
    else{
        new_image_array_d[idx] = (uint8_t)255 - image_array_d[idx];
    }


    __syncthreads();
}

int main(void)
{

    // Read the image
    uint8_t *image_array = get_image_array();

    // Allocate output
    uint8_t *new_image_array = (uint8_t *)malloc(M * N * C);

    uint8_t *image_array_d;
    uint8_t *new_image_array_d;

    // Invert
    cudaMalloc((void **)&image_array_d, M * N * C * sizeof(uint8_t));
    cudaMalloc((void **)&new_image_array_d, M * N * C * sizeof(uint8_t));

    cudaMemcpy(image_array_d, image_array, M *N * C * sizeof(uint8_t), cudaMemcpyHostToDevice);
    cudaMemcpy(new_image_array_d, new_image_array, M * N * C * sizeof(uint8_t),
        cudaMemcpyHostToDevice);

    invertImage<<<N*M*C/1024, 1024>>>(image_array_d, new_image_array_d);
    cudaDeviceSynchronize();

    cudaMemcpy(new_image_array, new_image_array_d, M * N * C * sizeof(uint8_t),
        cudaMemcpyDeviceToHost);
    // Save the image
    save_image_array(new_image_array);

    return 0;
}
```

Appendix D: Code part 2 (rrr...ggg...bbb... format)

```
/*
 * Code snippet for importing / exporting image data.
```

```
 *
 * To convert an image to a pixel map, run `convert <name>.<extension> <name>.ppm
 *
 */
#include <cstdint>  // Data types
#include <iostream> // File operations

#define M 960     // VR width
#define N 1280    // VR height
#define C 3       // Colors
#define OFFSET 16 // Header length

uint8_t *get_image_array(void)
{
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Noets:
     *  - Images data is flattened per color, column, row.
     *  - The first 3 data elements are the RGB components
     *  - The first 3*M data elements represent the firts row of the image
     *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M}, r_{1,0}, ..., b_{b,M}, ..., b_{N,
         M}
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile = fopen("./anna.ppm", "rb");
    if (imageFile == NULL)
    {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Initialize empty image array
    uint8_t *image_array = (uint8_t *)malloc(M * N * C * sizeof(uint8_t) + OFFSET);

    // Read the image
    fread(image_array, sizeof(uint8_t), M * N * C * sizeof(uint8_t) + OFFSET, imageFile);

    // Close the file
    fclose(imageFile);
    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}

void save_image_array(uint8_t *image_array)
{
    /*
     * Save the data of an (RGB) image as a pixel map.
     *
     * Parameters:
     *  - param1: The data of an (RGB) image as a 1D array
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile = fopen("./output_image_ex2_diff_format.ppm", "wb");
    if (imageFile == NULL)
    {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Configure the file
    fprintf(imageFile, "P6\n");          // P6 filetype
    fprintf(imageFile, "%d %d\n", M, N); // dimensions
    fprintf(imageFile, "255\n");         // Max pixel

    // Write the image
    fwrite(image_array, 1, M * N * C, imageFile);

    // Close the file
    fclose(imageFile);
}

```

```
__global__ void invertImage(uint8_t *image_array_d, uint8_t *new_image_array_d)
{
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if(idx < M*N){
        if(1 < idx < M*N - 1){
            new_image_array_d[idx] = (uint8_t) 0.1*image_array_d[idx-2] + 0.25 * image_array_d[
                idx -1] + 0.5 * image_array_d[idx] + 0.25 *image_array_d[idx + 1] + 0.1*
                image_array_d[idx + 2];
        }
    }else{
        new_image_array_d[idx] = (uint8_t)255 - image_array_d[idx];
    }


    __syncthreads();
}

int main(void)
{

    // Read the image
    uint8_t *image_array = get_image_array();

    uint8_t *reformat_array = (uint8_t *)malloc(M*N*C);
    int pixel_count = M*N;
    for(int i = 0; i < pixel_count; i++){
        reformat_array[i] = image_array[i * 3];
        reformat_array[i + pixel_count] = image_array[i * 3 + 1];
        reformat_array[i + 2 * pixel_count] = image_array[i * 3 + 2];
    }

    // Allocate output
    uint8_t *new_image_array = (uint8_t *)malloc(M * N * C);

    uint8_t *image_array_d;
    uint8_t *new_image_array_d;

    // Invert
    cudaMalloc((void **)&image_array_d, M * N * C * sizeof(uint8_t));
    cudaMalloc((void **)&new_image_array_d, M * N * C * sizeof(uint8_t));

    cudaMemcpy(image_array_d, reformat_array, M *N * C * sizeof(uint8_t), cudaMemcpyHostToDevice)
        ;
    cudaMemcpy(new_image_array_d, new_image_array, M * N * C * sizeof(uint8_t),
        cudaMemcpyHostToDevice);

    invertImage<<<N*M*C/1024, 1024>>>(image_array_d, new_image_array_d);
    cudaDeviceSynchronize();

    cudaMemcpy(new_image_array, new_image_array_d, M * N * C * sizeof(uint8_t),
        cudaMemcpyDeviceToHost);
    // Save the image

    uint8_t *new_rereformat_array = (uint8_t *)malloc(M*N*C);
    for(int i = 0; i < pixel_count; i++){
        new_rereformat_array[i * 3] = new_image_array[i];
        new_rereformat_array[i *3 + 1] = new_image_array[i + pixel_count];
        new_rereformat_array[i *3 + 2] = new_image_array[i + 2 * pixel_count];
    }
    save_image_array(new_rereformat_array);

    return 0;
}
```

References

[1] 'CUDA C++ Programming Guide'. Accessed: Feb. 24, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

[2] 'CUDA Grid-Stride Loops: What if you Have More Data Than Threads?'. Accessed: Feb. 24, 2025. [Online]. Available: https://alexminnaar.com/2019/08/02/grid-stride-loops.html